

- DSCI 100: Introduction to Data Science
- 2024S1 (Modified in 2026 Jan-Feb)
- Steven Lu (46989935)
- Heart Disease Project Report

Title:

Levels of Heart Disease by Exploring Resting Blood Pressure (in mm Hg on admission to the hospital), Maximum Heart Rate Achieved and Age

Introduction:

- Provide some relevant background information on the topic so that someone unfamiliar with it will be prepared to understand the rest of your report
- Clearly state the question you tried to answer with your project
- Identify and describe the dataset that was used to answer the question

The World Health Organization has estimated that 12 million deaths occur every year worldwide due to the heart diseases. Cardiovascular disease (CVD) is a kind of heart disease that affect the heart and the blood vessels and the way in which blood is pumped and circulated through the body. CVD results in several illness, disability, and death. Medical diagnosis of heart disease is a very important job (Soni et al., 2011). Many scholars undertook projects, algorithms and models to predict the probability of heart disease based on different attributes. Detrano et al. (1989) undertook a project that determine if CAD probabilities in study samples drawn from various ethnic populations with different clinical characteristics can be predicted by a probability algorithm derived from the clinical characteristics. They compared this algorithm with the algorithm of Diamond et al. They concluded that the algorithm is useful when applied to patients who have chest pain syndromes and intermediate disease prevalence. Shah et al. (2020) used the existing dataset from the Cleveland database of UCI repository of heart disease patients to predict the probability of developing heart disease using different algorithms, such as Naïve Bayes, decision tree, K-nearest neighbor, and random forest algorithm.

Now, we are given a modified version of the UCI Heart Disease dataset. We are given 14 out of 76 attributes, or variables to do a further data analysis for the heart disease. We are given lots of observations from 4 locations, which are Cleveland, Hungarian, Switzerland and VA Long Beach. Shah et al. (2020) used the dataset from the Cleveland database to perform data analysis of heart disease. I would like to use the dataset from the Hungarian, Cleveland, Switzerland and VA database, and then combine the datasets together in order to provide sufficient observations for models to train. In the 14 variables, 13 out of them are “feature” roles, which means that they are the predictors to predict a class. The “num” variable is the “target” role, which is the class I have to predict. Some of the variables are “integer” type, which means numeric-type variables that can be used for calculations. Some of the variables are “categorical” type, which is equivalent to “factor” type we learnt in the data science course.

The “num” variable is the class variable in the prediction process. It is the diagnosis of heart disease. In other words, it is the angiographic disease status. There are 2 values in the variable. Value 0 means less than 50% diameter of the vessel narrowing. Value 1 means more than 50% diameter of

the vessel narrowing. By using a K-nearest neighbour classification model, I would like to answer the following predictive question: For given 13 input features, will heart disease occur.

Methods & Results:

- Describe in written English the methods you used to perform your analysis from beginning to end that narrates the code the does the analysis.

In the UCI heart disease dataset, I would use all input features from the UCI dataset as the predictors of the classification. The reason why I used them as the predictor variables is that to provide sufficient input for more accurate diagnosis of heart disease.

For the data analysis, I would like to use classification model for the prediction. It would be done with the use of k-nearest neighbour algorithm as it does not require any specific shape assumption for the algorithm to perform well. To predict the heart disease, Shah et al. (2020) cited Pouriyeh et al. study that 83.16% accuracy was achieved with value K=9. For the percentage accuracy results of classification techniques, the accuracy from the testing set is 78.9% with K=2, and that from the training set is 90.8% with K=7 (Shah et al., 2020).

Here, I would first split the data into 80% training set and 20% testing set, then I would try to use the training set to create a tuning classification model and find the value of K which has the highest accuracy. After that, I would create the model again but using the k found previously, fit the model into the training data and predict the levels of heart disease using the testing data. Finally, I would find the accuracy of the k-nearest model by looking at the confusion matrix of the data.

Your report should include code which:

- Loads data

Before loading the data, libraries should be loaded so as to load the functions used for the data analysis.

```
In [1]: install.packages("tidyverse")
install.packages("tidymodels")
install.packages("repr") # needed for Jupyter notebook display options
install.packages("kknn")
```

```
package 'tidyverse' successfully unpacked and MD5 sums checked
```

```
The downloaded binary packages are in
```

```
  C:\Users\User\AppData\Local\Temp\Rtmp88ddow\downloaded_packages
```

```
package 'tidymodels' successfully unpacked and MD5 sums checked
```

```
The downloaded binary packages are in
```

```
  C:\Users\User\AppData\Local\Temp\Rtmp88ddow\downloaded_packages
```

```
Warning message:
```

```
"package 'repr' is in use and will not be installed"
```

```
package 'kknn' successfully unpacked and MD5 sums checked
```

```
Warning message:
"cannot remove prior installation of package 'kknn'"
Warning message in file.copy(savedcopy, lib, recursive = TRUE):
"problem copying C:\Program Files\R\R-4.5.1\library\00LOCK\kknn\libs\x64\kknn.dll to C:\Program Files\R\R-4.5.1\library\kknn\libs\x64\kknn.dll: Permission denied"
Warning message:
"restored 'kknn'"
```

The downloaded binary packages are in
C:\Users\User\AppData\Local\Temp\Rtmp88ddow\downloaded_packages

```
In [2]: library(tidyverse)
library(tidymodels)
library(repr)
options(repr.matrix.max.rows = 6)
library(kknn)
```

Warning message:
"package 'tidyverse' was built under R version 4.5.2"
Warning message:
"package 'tidyr' was built under R version 4.5.2"
Warning message:
"package 'purrr' was built under R version 4.5.2"
Warning message:
"package 'dplyr' was built under R version 4.5.2"
Warning message:
"package 'lubridate' was built under R version 4.5.2"

— Attaching core tidyverse packages — tidyverse 2.0.0 —

✓ dplyr	1.1.4	✓ readr	2.1.5
✓ forcats	1.0.0	✓ stringr	1.5.1
✓ ggplot2	3.5.2	✓ tibble	3.3.0
✓ lubridate	1.9.4	✓ tidyr	1.3.2
✓ purrr	1.2.1		

— Conflicts — tidyverse_conflicts() —

✗ dplyr::filter() masks stats::filter()
✗ dplyr::lag() masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors

Warning message:
"package 'tidymodels' was built under R version 4.5.2"

— Attaching packages — tidymodels 1.4.1 —

✓ broom	1.0.11	✓ rsample	1.3.1
✓ dials	1.4.2	✓ tailor	0.1.0
✓ infer	1.1.0	✓ tune	2.0.1
✓ modeldata	1.5.1	✓ workflows	1.3.0
✓ parsnip	1.4.1	✓ workflowsets	1.1.1
✓ recipes	1.3.1	✓ yardstick	1.3.2

Warning message:
"package 'broom' was built under R version 4.5.2"
Warning message:
"package 'dials' was built under R version 4.5.2"
Warning message:
"package 'infer' was built under R version 4.5.2"
Warning message:
"package 'modeldata' was built under R version 4.5.2"
Warning message:
"package 'parsnip' was built under R version 4.5.2"
Warning message:
"package 'recipes' was built under R version 4.5.2"
Warning message:
"package 'rsample' was built under R version 4.5.2"
Warning message:
"package 'tailor' was built under R version 4.5.2"

```
Warning message:
"package 'tune' was built under R version 4.5.2"
Warning message:
"package 'workflows' was built under R version 4.5.2"
Warning message:
"package 'workflowsets' was built under R version 4.5.2"
Warning message:
"package 'yardstick' was built under R version 4.5.2"
```

```
— Conflicts — tidymodels_conflicts() —
* scales::discard() masks purrr::discard()
* dplyr::filter() masks stats::filter()
* recipes::fixed() masks stringr::fixed()
* dplyr::lag() masks stats::lag()
* yardstick::spec() masks readr::spec()
* recipes::step() masks stats::step()

Warning message:
"package 'kknn' was built under R version 4.5.2"
```

Now we load the Hungarian, Cleveland, Switzerland and VA data which are comma-separated into the R program, so we use `read_csv()` function. Then, we combine the 4 datasets into 1 big dataset. Since there are no column names in the original table, we add `col_names = FALSE` parameter to indicate that.

```
In [44]: hungarian_original <- read_csv("data/heart_disease/processed.hungarian.data",
                                         col_names = FALSE)
cleveland_original <- read_csv("data/heart_disease/processed.cleveland.data",
                               col_names = FALSE)
switzerland_original <- read_csv("data/heart_disease/processed.switzerland.data",
                                 col_names = FALSE)
va_original <- read_csv("data/heart_disease/processed.va.data",
                       col_names = FALSE)

combined <- rbind(hungarian_original, cleveland_original, switzerland_original, va_original)
combined
```

Rows: 294 Columns: 14

— Column specification —

Delimiter: ","

chr (9): X4, X5, X6, X7, X8, X9, X11, X12, X13

dbl (5): X1, X2, X3, X10, X14

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

Rows: 303 Columns: 14

— Column specification —

Delimiter: ","

chr (2): X12, X13

dbl (12): X1, X2, X3, X4, X5, X6, X7, X8, X9, X10, X11, X14

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

Rows: 123 Columns: 14

— Column specification —

Delimiter: ","

chr (9): X4, X6, X7, X8, X9, X10, X11, X12, X13

dbl (5): X1, X2, X3, X5, X14

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

Rows: 200 Columns: 14

— Column specification —

Delimiter: ","

chr (9): X4, X5, X6, X8, X9, X10, X11, X12, X13

dbl (5): X1, X2, X3, X7, X14

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

A spec_tbl_df: 920 × 14

X1	X2	X3	X4	X5	X6	X7	X8	X9	X10	X11	X12	X13	X14
<dbl>	<dbl>	<dbl>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>	<dbl>
28	1	2	130	132	0	2	185	0	0	?	?	?	0
29	1	2	120	243	0	0	160	0	0	?	?	?	0
29	1	2	140	?	0	0	170	0	0	?	?	?	0
:	:	:	:	:	:	:	:	:	:	:	:	:	:
55	1	4	122	223	1	1	100	0	0	?	?	6	2
58	1	4	?	385	1	2	?	?	?	?	?	?	0
62	1	2	120	254	0	2	93	1	0	?	?	?	1

- Wrangles and cleans the data to the format necessary for the planned analysis

First of all, we assign the corresponding column names to the data frame based on the UCI website. Then, we found that the data type of most of the columns are incorrect, based on the UCI website, we should mutate every column so that they have the correct data type. After that, we named different diagnosis of heart disease in the "num" column for easier interpretation.

```
In [45]: colnames(combined) <- c('age','sex','cp','trestbps','chol','fbs','restcg','thalach','exang','oldpeak','slope','ca','thal','num')
combined <- combined |>
  mutate(age = as.integer(age), sex = as_factor(sex), cp = as_factor(cp),
         trestbps = as.integer(trestbps), chol = as.integer(chol),
         fbs = as_factor(fbs), restcg = as_factor(restcg),
         thalach = as.integer(thalach), exang = as_factor(exang),
         oldpeak = as.integer(oldpeak), slope = as_factor(slope),
         ca = as.integer(ca), thal = as_factor(thal),
         num = as_factor(num)) |>
  mutate(sex = fct_recode(sex, "male" = "1", "female" = "0"),
         num = fct_recode(num, "Absent" = "0", "Present" = "1", "Present" = "2", "Present" = "3", "Present" = "4"))
combined
```

Warning message:
 "There were 5 warnings in `mutate()``.
 The first warning was:
 i In argument: `trestbps = as.integer(trestbps)`.
 Caused by warning:
 ! NAs introduced by coercion
 i Run `dplyr::last\_dplyr\_warnings()` to see the 4 remaining warnings."

A tibble: 920 × 14

age	sex	cp	trestbps	chol	fbs	restcg	thalach	exang	oldpeak	slope	ca	thal	num
<int>	<fct>	<fct>	<int>	<int>	<fct>	<fct>	<int>	<fct>	<int>	<fct>	<int>	<fct>	<fct>
28	male	2	130	132	0	2	185	0	0	?	NA	?	Absent
29	male	2	120	243	0	0	160	0	0	?	NA	?	Absent
29	male	2	140	NA	0	0	170	0	0	?	NA	?	Absent
:	:	:	:	:	:	:	:	:	:	:	:	:	:
55	male	4	122	223	1	1	100	0	0	?	NA	6	Present
58	male	4	NA	385	1	2	NA	?	NA	?	NA	?	Absent
62	male	2	120	254	0	2	93	1	0	?	NA	?	Present

We handle missing values with simple imputation. For numerical features, we use imputation with median. For categorial features, we use imputation with mode.

```
In [46]: colSums(is.na(combined))
```

age: 0 sex: 0 cp: 0 trestbps: 59 chol: 30 fbs: 0 restcg: 0 thalach: 55 exang: 0 oldpeak: 62 slope: 0 ca: 611 thal: 0 num: 0

```
In [47]: combined$trestbps[is.na(combined$trestbps)] <- median(combined$trestbps, na.rm = TRUE)
combined$chol[is.na(combined$chol)] <- median(combined$chol, na.rm = TRUE)
combined$thalach[is.na(combined$thalach)] <- median(combined$thalach, na.rm = TRUE)
combined$oldpeak[is.na(combined$oldpeak)] <- median(combined$oldpeak, na.rm = TRUE)
combined$ca[is.na(combined$ca)] <- median(combined$ca, na.rm = TRUE)
```

```
In [48]: combined
```


A tibble: 920 × 14

age	sex	cp	trestbps	chol	fbs	restcg	thalach	exang	oldpeak	slope	ca	thal	num
<int>	<fct>	<fct>	<int>	<dbl>	<fct>	<fct>	<int>	<fct>	<dbl>	<fct>	<int>	<fct>	<fct>
28	male	2	130	132	0	2	185	0	0	?	0	?	Absent
29	male	2	120	243	0	0	160	0	0	?	0	?	Absent
29	male	2	140	223	0	0	170	0	0	?	0	?	Absent
:	:	:	:	:	:	:	:	:	:	:	:	:	:
55	male	4	122	223	1	1	100	0	0	?	0	6	Present
58	male	4	130	385	1	2	140	?	0	?	0	?	Absent
62	male	2	120	254	0	2	93	1	0	?	0	?	Present

```
In [49]: colSums(combined == "?", na.rm = TRUE)
combined[combined == "?"] <- NA
colSums(is.na(combined))
```

age: 0 sex: 0 cp: 0 trestbps: 0 chol: 0 fbs: 90 restcg: 2 thalach: 0 exang: 55 oldpeak: 0 slope: 309 ca: 0 thal: 486 num: 0

age: 0 sex: 0 cp: 0 trestbps: 0 chol: 0 fbs: 90 restcg: 2 thalach: 0 exang: 55 oldpeak: 0 slope: 309 ca: 0 thal: 486 num: 0

```
In [54]: mode_val <- names(sort(table(combined$fbs), decreasing = TRUE))[1]
combined$fbs[is.na(combined$fbs)] <- mode_val
mode_val <- names(sort(table(combined$restcg), decreasing = TRUE))[1]
combined$restcg[is.na(combined$restcg)] <- mode_val
mode_val <- names(sort(table(combined$exang), decreasing = TRUE))[1]
combined$exang[is.na(combined$exang)] <- mode_val
mode_val <- names(sort(table(combined$slope), decreasing = TRUE))[1]
combined$slope[is.na(combined$slope)] <- mode_val
mode_val <- names(sort(table(combined$thal), decreasing = TRUE))[1]
combined$thal[is.na(combined$thal)] <- mode_val

combined
colSums(is.na(combined))
```

A tibble: 920 × 14

age	sex	cp	trestbps	chol	fbs	restcg	thalach	exang	oldpeak	slope	ca	thal	num
<int>	<fct>	<fct>	<int>	<dbl>	<fct>	<fct>	<int>	<fct>	<dbl>	<fct>	<int>	<fct>	<fct>
28	male	2	130	132	0	2	185	0	0	2	0	3.0	Absent
29	male	2	120	243	0	0	160	0	0	2	0	3.0	Absent
29	male	2	140	223	0	0	170	0	0	2	0	3.0	Absent
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
55	male	4	122	223	1	1	100	0	0	2	0	6	Present
58	male	4	130	385	1	2	140	0	0	2	0	3.0	Absent
62	male	2	120	254	0	2	93	1	0	2	0	3.0	Present

age: 0 sex: 0 cp: 0 trestbps: 0 chol: 0 fbs: 0 restcg: 0 thalach: 0 exang: 0 oldpeak: 0 slope: 0 ca: 0 thal: 0 num: 0

```
In [55]: dataset <- combined
dataset
```

A tibble: 920 × 14

age	sex	cp	trestbps	chol	fbs	restcg	thalach	exang	oldpeak	slope	ca	thal	num
<int>	<fct>	<fct>	<int>	<dbl>	<fct>	<fct>	<int>	<fct>	<dbl>	<fct>	<int>	<fct>	<fct>
28	male	2	130	132	0	2	185	0	0	2	0	3.0	Absent
29	male	2	120	243	0	0	160	0	0	2	0	3.0	Absent
29	male	2	140	223	0	0	170	0	0	2	0	3.0	Absent
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
55	male	4	122	223	1	1	100	0	0	2	0	6	Present
58	male	4	130	385	1	2	140	0	0	2	0	3.0	Absent
62	male	2	120	254	0	2	93	1	0	2	0	3.0	Present

- Performs a summary of the data set that is relevant for exploratory data analysis related to the planned analysis

There are a lot of ways to summarize the data. For example, we can find the number of cases that has <50% Diameter Narrowing and >50% Diameter Narrowing. We can also find the mean "trestbps", "thalach" and "age" and their median grouped by "num" for comparison.

```
In [6]: dataset_countclass <- dataset |>
        group_by(num) |>
        summarize(count = n())

dataset_countclass

dataset_classstat <- dataset |>
        group_by(num) |>
        summarize(mean_trestbps = mean(trestbps, na.rm = TRUE),
                  mean_thalach = mean(thalach, na.rm = TRUE),
                  mean_age = mean(age, na.rm = TRUE),
                  median_trestbps = median(trestbps, na.rm = TRUE),
                  median_thalach = median(thalach, na.rm = TRUE),
                  median_age = median(age, na.rm = TRUE))

dataset_classstat
```

A tibble: 2 × 2

num	count
<fct>	<int>
Absent	411
Present	509

A tibble: 2 × 7

num	mean_trestbps	mean_thalach	mean_age	median_trestbps	median_thalach	median_age
<fct>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<int>
Absent	129.9130	148.8005	50.54745	130	151	51
Present	133.9787	128.2616	55.90373	130	128	57

Based on the results, we can see that there are more cases having <50% Diameter Narrowing than >50% Diameter Narrowing. Based on mean and the median, we can see that for >50% Diameter Narrowing, "trestbps" and "age" are generally higher, but "thalach" is generally lower.

- Creates a visualization of the dataset that is relevant for exploratory data analysis related to the planned analysis

For data visualization, we could first use the bar graph to compare the number of cases that has <50% Diameter Narrowing and >50% Diameter Narrowing. Then, we can find the distribution of "age", "thalach" and "trestbps" and compare each of the distribution whose "num" is ">50% Diameter Narrowing" with that whose "num" is "<50% Diameter Narrowing". This might be helpful for data analysis. We use options() function to adjust the plot size to suitable size. A width 7 units and height 6 units would be a suitable size because, when we export or print the report document, the graph won't be "cropped" because the size is too big.

```
In [7]: options(repr.plot.width = 7, repr.plot.height = 6)

dataset_countclass_plot <- ggplot(dataset_countclass, aes(x = num, y = count)) +
  geom_bar(stat = "identity") +
  labs(x = "Level of Heart Disease", y = "Number of Observations",
       title = "Number of Observations of Different Levels of Heart Disease") +
  theme(text = element_text(size = 14))

dataset_countclass_plot

dataset_ageplot <- ggplot(dataset, aes(x = age)) +
  geom_histogram(binwidth = 1.0) +
  labs(x = "Age", y = "Number of Observations",
       title = "Age Distribution") +
  facet_grid(cols = vars(num)) +
  theme(text = element_text(size = 14))

dataset_ageplot

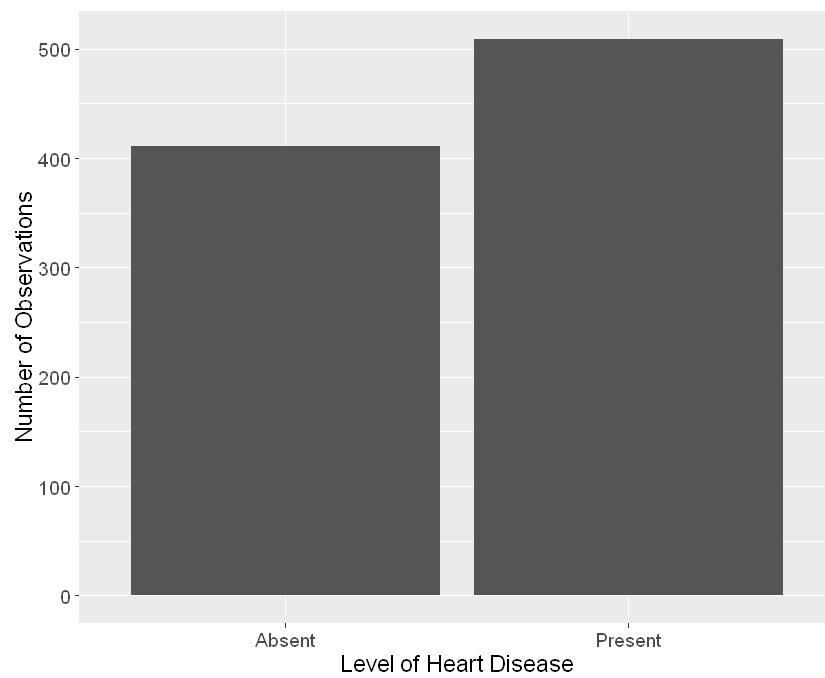
dataset_trestbpsplot <- ggplot(dataset, aes(x = trestbps)) +
  geom_histogram(binwidth = 3.0) +
  labs(x = "Resting Blood Pressure (mm Hg)", y = "Number of Observations",
       title = "Resting Blood Pressure Distribution") +
  facet_grid(cols = vars(num)) +
  theme(text = element_text(size = 13))

dataset_trestbpsplot

dataset_thalachplot <- ggplot(dataset, aes(x = thalach)) +
  geom_histogram(binwidth = 3.0) +
  labs(x = "Maximum Heart Rate Achieved", y = "Number of Observations",
       title = "Maximum Heart Rate Achieved Distribution") +
  facet_grid(cols = vars(num)) +
  theme(text = element_text(size = 13))

dataset_thalachplot
```

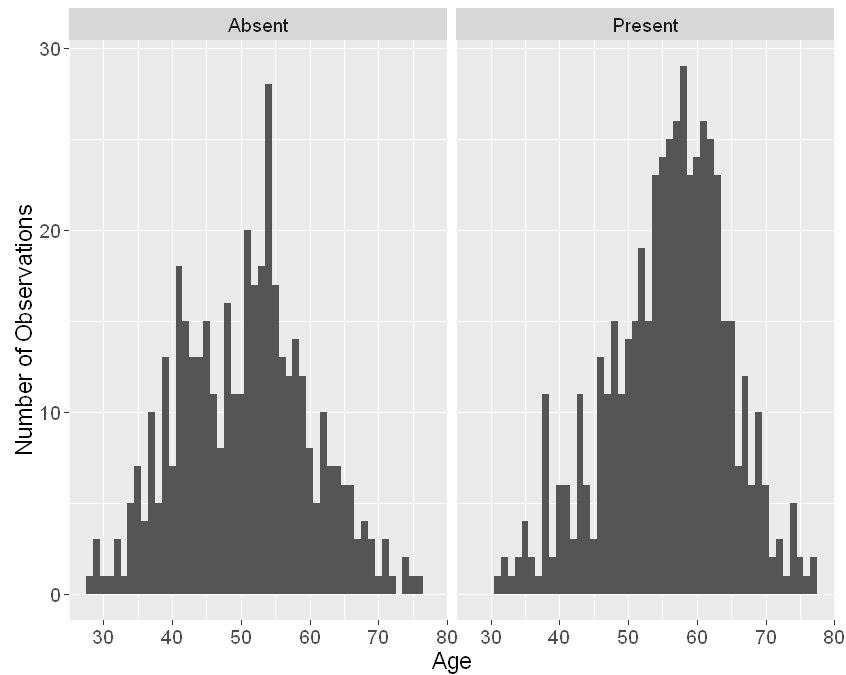
Number of Observations of Different Levels of Heart Disease



Warning message:

"Removed 59 rows containing non-finite outside the scale range (``stat_bin()``)."

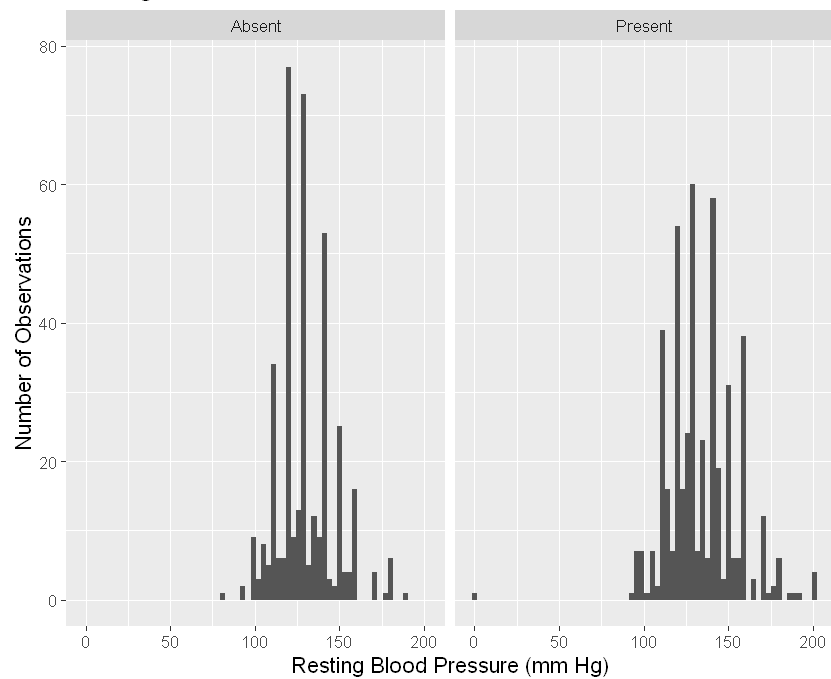
Age Distribution



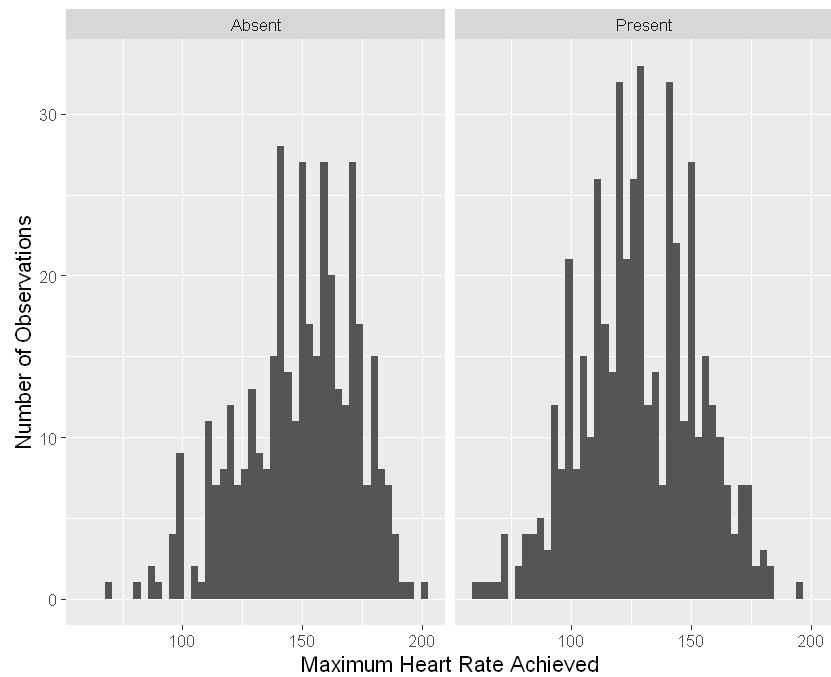
Warning message:

"Removed 55 rows containing non-finite outside the scale range (``stat_bin()``)."

Resting Blood Pressure Distribution



Maximum Heart Rate Achieved Distribution



Based on the histograms, we can see the age distribution among different levels of heart disease are similar, as they are both centered at 54. The resting blood pressure distributions are different. For "<50% Diameter Narrowing", the distribution is centered at 120mm Hg, while the distribution

is centered at 141mm Hg. The maximum heart rate distributions are different. For "<50% Diameter Narrowing", the distribution is centered at 138, while the distribution is centered at 150.

- Performs the data analysis

We will start doing classification for data analysis. There are 2 big steps involved in classification. The first one is to predict k, which involves several small steps:

1. Split the Hungarian data
2. Build the model with tune()
3. Build the recipe for the model
4. Build a cross-validation fold
5. Create a sequence tibble
6. Create a workflow and add tune_grid()
7. Find k with the highest accuracy

```
In [56]: set.seed(1234)

split <- initial_split(dataset, prop = 0.8, strata = num)
train <- training(split)
test <- testing(split)

knn_spec <- nearest_neighbor(
  neighbors = tune(),
  weight_func = tune(),
  dist_power = tune()
) |>
  set_engine("kknn") |>
  set_mode("classification")
```

```
In [57]: train
```

A tibble: 735 × 14

age	sex	cp	trestbps	chol	fbs	restcg	thalach	exang	oldpeak	slope	ca	thal	num
<int>	<fct>	<fct>	<int>	<dbl>	<fct>	<fct>	<int>	<fct>	<dbl>	<fct>	<int>	<fct>	<fct>
29	male	2	120	243	0	0	160	0	0	2	0	3.0	Absent
30	female	1	170	237	0	1	170	0	0	2	0	6	Absent
32	female	2	105	198	0	0	165	0	0	2	0	3.0	Absent
:	:	:	:	:	:	:	:	:	:	:	:	:	:
54	female	4	127	333	1	1	154	0	0	2	0	3.0	Present
55	male	4	122	223	1	1	100	0	0	2	0	6	Present
62	male	2	120	254	0	2	93	1	0	2	0	3.0	Present

```
In [73]: dataset_recipe <- recipe(num ~ age + sex + cp + trestbps + chol + fbs + restcg + thalach + exang + oldpeak + slope + ca + thal,
  data = train) |>
  step_impute_median(all_numeric_predictors()) |>
  step_impute_mode(all_nominal_predictors()) |>
  step_dummy(all_nominal_predictors()) |>
  step_center(all_numeric_predictors()) |>
  step_scale(all_numeric_predictors())

dataset_fold <- vfold_cv(data = train, v = 5, repeats = 1, strata = num)

gridvals <- expand_grid(
  neighbors = seq(1, 20, 1),
  weight_func = c("rectangular", "optimal", "triangular", "epanechnikov"),
  dist_power = c(1, 2)
)

# gridvals <- tibble(neighbors = seq(from = 1, to = 50, by = 1))

training_workflow <- workflow() |>
  add_recipe(dataset_recipe) |>
  add_model(knn_spec) |>
  tune_grid(resamples = dataset_fold, grid = gridvals)

dataset_findk <- training_workflow |>
  collect_metrics() |>
  filter(.metric == "accuracy") |>
  filter(mean == max(mean)) |>
  slice_head(n = 1)
```



```
dataset_findk
```

```
→ A | warning: ! The following columns have zero variance so scaling cannot be used: fbs_X., restcg_X., and exang_X..  
i Consider using ?step_zv (`?recipes::step_zv()`) to remove those columns before normalizing.
```

```
There were issues with some computations A: x1  
There were issues with some computations A: x2  
There were issues with some computations A: x3  
There were issues with some computations A: x4  
There were issues with some computations A: x5  
There were issues with some computations A: x5
```

A tibble: 1 × 9

neighbors	weight_func	dist_power	.metric	.estimator	mean	n	std_err	.config
<dbl>	<chr>	<dbl>	<chr>	<chr>	<dbl>	<int>	<dbl>	<chr>
19	rectangular	1	accuracy	binary	0.8270934	5	0.01270682	pre0_mod145_post0

After testing with different values of prop, weight_func and v, we found that with prop = 0.75, weight_func = “optimal” and v = 6, the accuracy is 74.4%, which is the highest so far. Now the second big step is to actually build the model and do the prediction using the test data. The second big step involves several small steps:

1. Build a model with a given k and weight_func
2. Use the same recipe
3. Create the workflow and fit the training data into the workflow
4. Predict the testing data
5. Find the metrics and the confusion matrix

```
In [74]: best_neighbors <- dataset_findk %>% pull(neighbors)  
best_weight <- dataset_findk %>% pull(weight_func)  
best_distance <- dataset_findk %>% pull(dist_power)  
  
best_neighbors  
best_weight  
best_distance
```

```
19  
'rectangular'  
1
```

```
In [75]: set.seed(1234)

knn_spec_givenk <- nearest_neighbor(weight_func = best_weight, neighbors = best_neighbors, dist_power = best_distance) |>
  set_engine("kknn") |>
  set_mode("classification")

dataset_fit <- workflow() |>
  add_recipe(dataset_recipe) |>
  add_model(knn_spec_givenk) |>
  fit(data = train)

dataset_predict <- predict(dataset_fit, test) |>
  bind_cols(test)

dataset_metrics <- dataset_predict |>
  metrics(truth = num, estimate = .pred_class)
dataset_metrics

dataset_confmat <- dataset_predict |>
  conf_mat(truth = num, estimate = .pred_class)
dataset_confmat
```

Warning message:

"! The following columns have zero variance so scaling cannot be used: fbs_X., restcg_X., and exang_X..
 i Consider using ?step_zv (`?recipes::step\_zv()`) to remove those columns before normalizing."

A tibble: 2 × 3

.metric	.estimator	.estimate
<chr>	<chr>	<dbl>
accuracy	binary	0.8216216
kap	binary	0.6422502

Truth		
Prediction	Absent	Present
Absent	70	20
Present	13	82

- Creates a visualization of the analysis

One way to create a visualization is to plot a line graph of accuracy estimate against number of neighbors.

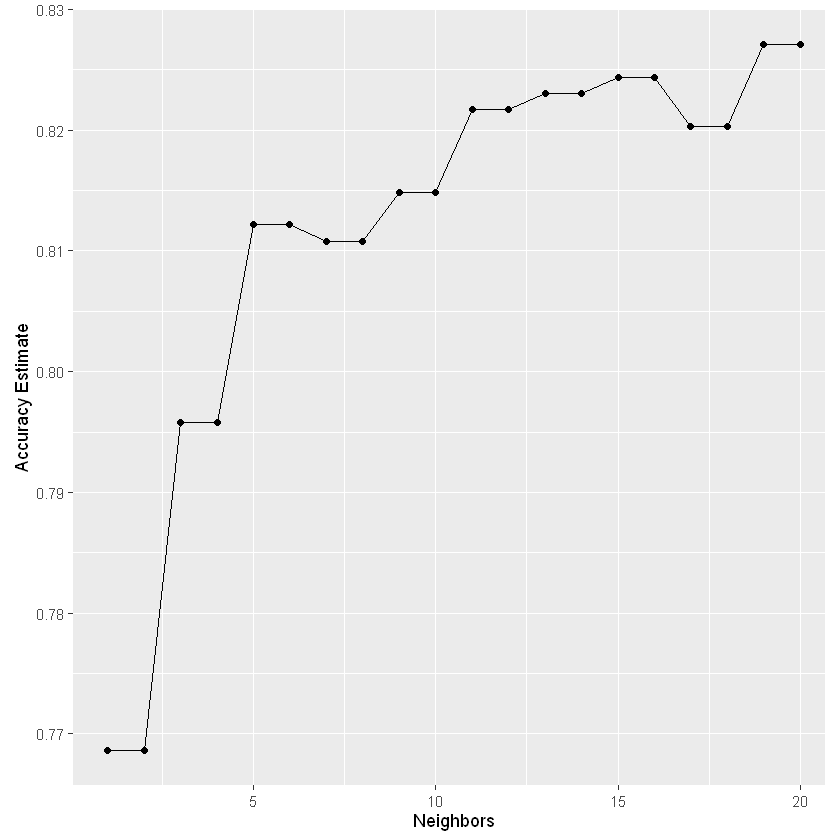
```
In [76]: accuracies <- training_workflow |>
  collect_metrics() |>
  filter(.metric == "accuracy", weight_func == best_weight, dist_power == best_distance)
```

accuracies

```
accuracy_versus_k <- ggplot(accuracies, aes(x = neighbors, y = mean)) +  
  geom_point() + geom_line() +  
  labs(x = "Neighbors", y = "Accuracy Estimate")  
accuracy_versus_k
```

A tibble: 20 × 9

neighbors	weight_func	dist_power	.metric	.estimator	mean	n	std_err	.config
<dbl>	<chr>	<dbl>	<chr>	<chr>	<dbl>	<int>	<dbl>	<chr>
1	rectangular	1	accuracy	binary	0.7686156	5	0.01426511	pre0_mod001_post0
2	rectangular	1	accuracy	binary	0.7686156	5	0.01426511	pre0_mod009_post0
3	rectangular	1	accuracy	binary	0.7957996	5	0.01089481	pre0_mod017_post0
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
18	rectangular	1	accuracy	binary	0.8202720	5	0.01477705	pre0_mod137_post0
19	rectangular	1	accuracy	binary	0.8270934	5	0.01270682	pre0_mod145_post0
20	rectangular	1	accuracy	binary	0.8270934	5	0.01270682	pre0_mod153_post0



Discussion:

- Summarize what you found
- Discuss whether this is what you expected to find?
- Discuss what impact could such findings have?
- Discuss what future questions could this lead to?

After performing data analysis, I found that when the proportion of training set is 80%, using "rectangular" as weight_func and using 5-fold cross-validation, the predicted accuracy is 82.7%. When I predict the testing dataset, the accuracy is 82.2%, which is not ideal in the medical diagnosis situation. If I consider "present" as correct positive prediction, the precision is $82/(82+20)100\% = 80.4\%$ and the recall is $82/(82+13)100\% = 86.3\%$. If I consider "absent", the precision is $70/(70+13)100\% = 84.3\%$ and the recall is $70/(70+20)100\% = 77.8\%$, which seems really bad. By plotting a line graph of accuracy estimate against number of neighbors, I can see that the accuracy attains maximum when the value of k reaches 19, then the accuracy has a decreasing trend with an increasing k, showing that a higher k doesn't mean higher accuracy.

Looking at the findings, it is a bit beyond my expectations. When I am doing k-nn classification in previous worksheets or tutorial exercises, I got the accuracy really high accuracy such that the accuracy is nearly 90% with a few predictors used. I expect k-nn classification would give a similar accuracy when it is performed in heart disease situation because the situations are similar. However, when I am doing k-nn classification by

ourselves in a new situation, I found that the accuracy of k-nn classification is a bit lower than what I expected. This makes me realised that k-nn is not an ideal model for every situation.

If there is a set of values for v (number of folds), training proportion, weight_func, number of predictors, predictors chosen, etc. that gives the highest accuracy of k-nn, then it is possible that the accuracy of k-nn in heart disease diagnosis reaches 90% or higher. So far there is no scholars found the accuracy of k-nn in heart disease diagnosis exceed 90.8%. I think k-nn classification is not ideal for heart disease diagnosis, it might be because of its simplicity. If the accuracy of k-nn is high enough, the positive impact is that it would be more convenient and fast to diagnose a heart disease because you just need to enter the required information (i.e. predictors) and the diagnosis will be done by just a machine. However, medical diagnosis requires a very high accuracy. Otherwise, if the accuracy is not high enough, the patients would be treated wrongly, which would cause severe illness and even death. Ultimately this will become a disaster since many patients would be treated wrongly.

Doing the data analysis, I would like to ask questions further to myself: Are there other models to classify a patient the level of heart disease more accurately? What predictors, what parameters should be used to help the model attains its maximum accuracy? I am looking forward to answering these in future studies.

References

- At least 2 citations of literature relevant to the project (format is your choice, just be consistent across the references).
- Make sure to cite the source of your data as well.

Detrano, R., Janosi, A., Steinbrunn, W., Pfisterer M., Schmid J., Sandhu S., Guppy K., Lee S., Froelicher V. (1989). International Application of a New Probability Algorithm for the Diagnosis of Coronary Artery Disease. The American Journal of Cardiology, 64(5), 304-310.

[https://doi.org/10.1016/0002-9149\(89\)90524-9](https://doi.org/10.1016/0002-9149(89)90524-9)

Janosi, A., Steinbrunn, W., Pfisterer, M., Detrano, R. (1988). Heart Disease. UC Irving Machine Learning Repository.

<https://archive.ics.uci.edu/dataset/45/heart+disease>

Shah, D., Patel, S. & Bharti, S.K. (2020). Heart Disease Prediction using Machine Learning Techniques. SN Computer Science, 1(345), 1-6.

<https://doi.org/10.1007/s42979-020-00365-y>

Soni, J., Ansari, U., Sharma, D. & Soni, S. (2011). Predictive Data Mining For Medical Diagnosis: An Overview Of Heart Disease Prediction.

International Journal of Computer Applications, 17(8), 43-48. <https://doi.org/10.5120/2237-2860>